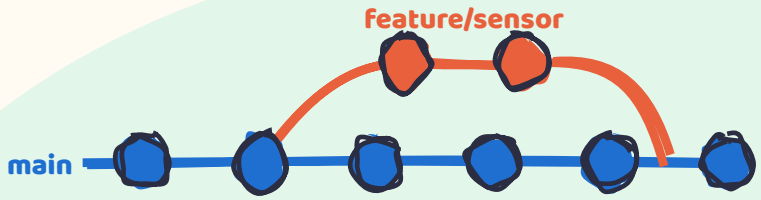
Tech Leaders · Class 22

Git & GitHub for Engineering Teams

Never lose work. Never overwrite a teammate.

Commits | Workflow | Branches | Pull requests



Did you know?
Linus Torvalds wrote Git in 2005 to manage the Linux kernel, a project with thousands of developers. Now almost all software teams use it.

LEARN

Let's think...

final_v2_REAL_final_fixed.ino?

Team chaos

Three teammates edit the robot code. Someone emails a version, someone copies a USB, someone 'fixes' the motor code and breaks the sensor. Which file is the latest? Who changed what? Can we get yesterday's working version back?



Use USBs?

One person codes?

Use Git!

Today: your team puts a robot project under **Git**, works on **branches**, resolves a **conflict** and merges **pull requests** on GitHub.

Learning objectives

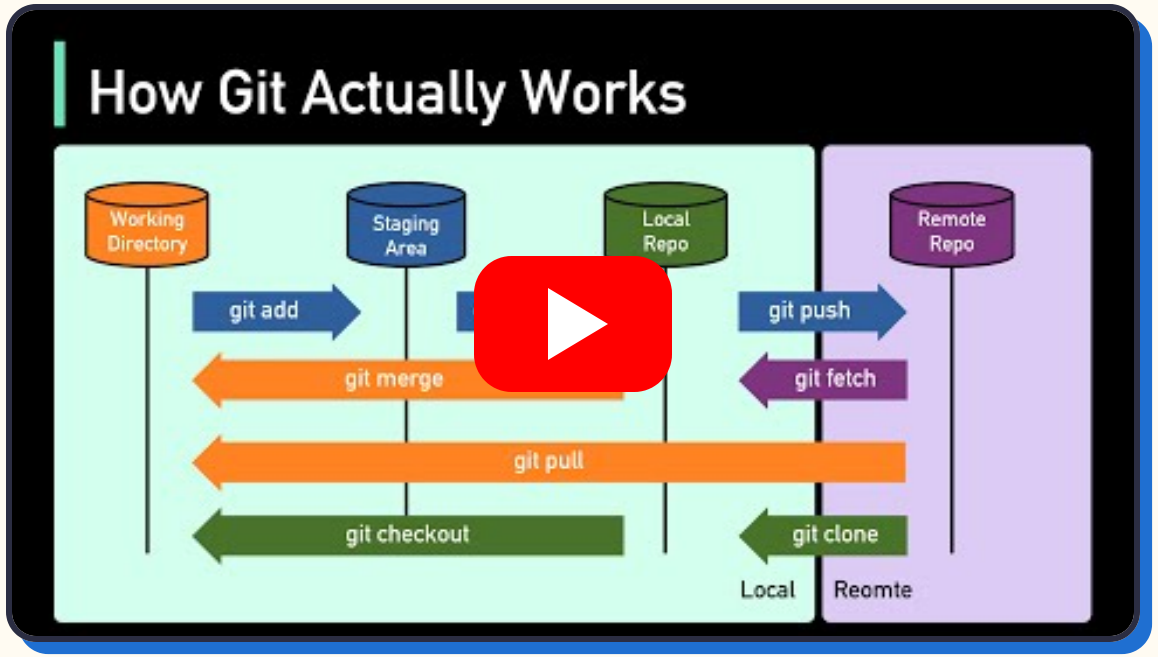
By the end of this class I can...

1	**Why version control:** explain **commits** as saved snapshots with messages, and why teams need history	**Proof:** I can find and undo yesterday's change
2	**Local workflow:** use **init, status, add, commit, log, diff** on a real project	**Proof:** my repo has 5+ meaningful commits
3	**Branches + merges:** work on a **branch**, **merge** it, and fix a **merge conflict**	**Proof:** I resolved a conflict without losing anyone's work
4	**Team on GitHub:** **push**, **pull**, and review a teammate's **pull request** safely	**Proof:** my team merged a reviewed PR into main

LEARN

Objective 1 · Why version control · video

Video time!



How Git Works: Explained in 4 Minutes

ByteByteGo · about 4 minutes · youtu.be/e9lnsKot_SQ

Watch for...

1. What is a **commit**?
2. What are **branches** for?
3. How do Git and GitHub differ?

Scan to watch

After the video: Think of a group project that went wrong. How would Git have helped?

Objective 1 · Why version control

Commits: snapshots with a story

The idea

A **repository** (repo) is a project folder that Git watches. A **commit** is a saved **snapshot** of the files, plus a **message**, an author and a time.

The history is a chain of commits. You can see **what** changed (diff), **who** and **why** (message), and go back to any point.

Good messages say **why**: 'Fix motor reversing when IR sees white' beats 'update'.

Example: save points in a game



Before a boss fight you save. If it goes badly, you reload. Commits are save points for code, with a note saying what each save was for.

LEARN

Objective 2 · Local workflow

The three areas: working, staging, repo

The idea

Working folder: files you are editing.
Staging area: changes you've chosen for the next commit (**git add**).
Repository: committed history (**git commit**).

Daily loop: edit → **git status** → **git diff** → **git add** file → **git commit -m** "why" → repeat.

Commit **small and often**: one idea per commit.

Example: packing a parcel



You pick items from your room (working), put chosen ones in the box (staging), then seal and label it (commit). Only sealed boxes are in the record.

LEARN

Objective 3 · Branches + merges

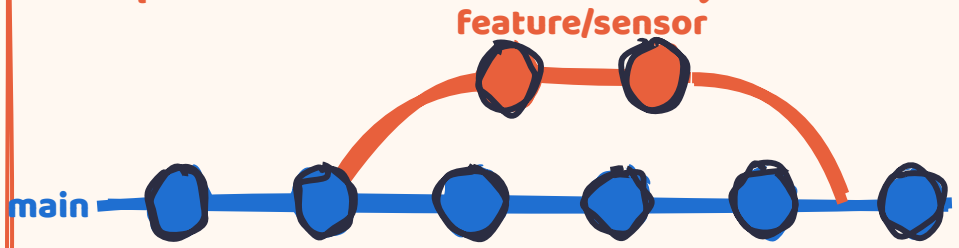
Branches, merges and conflicts

The idea

A **branch** is a parallel line of work: **main** stays working while you build **feature/sensor**. When it's ready, **merge** it back.

A **merge conflict** happens when two branches changed the **same lines**. Git marks them:
<<<<<<< HEAD ... ======= ... >>>>>>> feature
You choose (or combine) the right code, delete the markers, add, commit. Conflicts are normal, not disasters.

Example: two editors on one essay



Two students edit the same sentence differently. Someone has to decide which version stays, or write a better combined one. Git just shows you exactly where.

LEARN

Objective 4 · Team on GitHub

GitHub: the team's shared repo

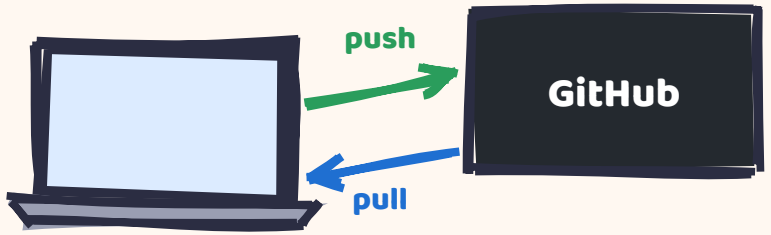
The idea

GitHub hosts a copy (**remote**, called origin) everyone shares.
git push: send your commits up. **git pull**: get teammates' commits down.
Pull **before** you start work.

Pull request (PR): 'please merge my branch'. Teammates **review** the diff, comment, request changes, approve.

.gitignore keeps junk (build files) out. **Never commit passwords or API keys**: public repos are searched by bots.

Example: a teacher checking work



Before your project goes on the class wall, a classmate checks it and suggests fixes. A pull request is that check for code.

Real world

Git runs the software world



Open source

Linux, Python and Arduino are built by thousands on Git



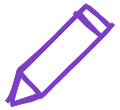
Every tech company

Code review via pull requests is standard practice



Your portfolio

Universities and employers look at GitHub profiles



Not just code

Writers, scientists and designers version their work too



What would you want an employer to see in your GitHub profile in 3 years?

Pakistani contributors

Pakistani developers contribute to major open-source projects; you can too, starting with docs and small fixes.

LEARN

Quick quiz

Git check

Question	Answer
1. What does git add do?	
2. Good or bad message: 'stuff'?	
3. Two branches changed the same line. What happens?	
4. What should you do before starting work in a team?	
5. Can you commit your Wi-Fi password? Why not?	

Answers

Cover! 1: stages changes for the next commit · 2: bad: say why · 3: merge conflict: you choose · 4: git pull · 5: no: history is permanent and repos get searched

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human Git
2. Our Git kit
3. Your first repo (terminal)
4. Branch, push, merge, resolve
5. Team sprint on GitHub
6. Repo health check
7. Team engineer challenges

MAKE

Class game

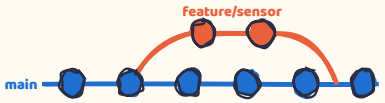
Human Git

How to play

1. Paper 'repo': a story on a strip, one sentence per sticky note (commits) on the wall: main.
2. Team A takes a copy (branch) and adds 2 sentences. Team B branches too and edits sentence 3.
3. Merge Team A (easy). Merge Team B: sentence 3 was changed by both? Conflict!
4. The class resolves it by choosing or combining. Write a 'commit message' explaining why.

Then discuss

Why did small, clear changes make merging easier?



Big idea

Branches let people work in parallel; merges bring work together.

MAKE

Build it

Our Git kit

Git
git-scm.com (check: git --version)

GitHub account
13+ (with parent permission); or class org

VS Code
Built-in Source Control panel

A real project
Your robot code from earlier classes

Team of 3
Owner, two contributors

Secrets file
secrets.h listed in .gitignore

First time only: git config --global user.name "Your Name" and user.email (use your GitHub no-reply email for privacy).

Privacy
Don't put your full name, school address or phone in public repos. Keep repos private until you're sure.

MAKE

Objective 2 · Local workflow · Build step 1 of 3

Your first repo (terminal)

```
# 1. start tracking a project
cd robot-car
git init
git status                      # what changed?

# 2. first snapshot
echo "secrets.h" >> .gitignore
git add .gitignore car.ino README.md
git commit -m "Add line-follower code and README"

# 3. daily loop
git diff                        # see exact changes
git add car.ino
git commit -m "Slow down on sharp turns to stop overshoot"
git log --oneline               # history

# 4. oops: get back the last committed version of a file
git restore car.ino
```

status

Run git status constantly: it tells you exactly what Git sees.

Small commits

One idea per commit makes history readable and undo easy.

restore

Throws away uncommitted changes to that file. Be sure!

MAKE

Objective 3 · Branches + merges · Build step 2 of 3

Branch, push, merge, resolve

```
# teammate B: work on a branch
git switch -c feature/ir-sensor
# ...edit, add, commit...
git push -u origin feature/ir-sensor     # then open a Pull Request on GitHub

# owner: update and merge
git switch main
git pull
git merge feature/ir-sensor
# CONFLICT in car.ino?  open it:
#   <<<<<<< HEAD
#   int speed = 150;
#   =======
#   int speed = 120;
#   >>>>>>> feature/ir-sensor
# choose/combine, delete the markers, then:
git add car.ino
git commit -m "Merge IR sensor; keep speed 120 for sharper turns"
git push
```

switch -c

Creates a new branch and moves onto it.

Conflict

Git never guesses: it stops and asks you. Talk to your teammate!

PR first

In real teams, merging happens on GitHub after review.

MAKE

Objective 4 · Build step 3 of 3

Team sprint on GitHub

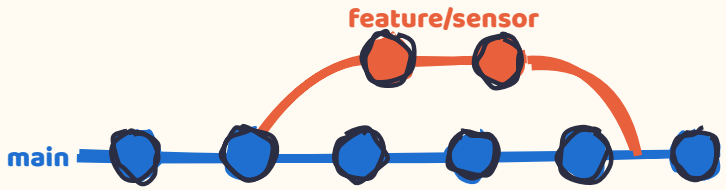
a Owner creates a **private** repo, pushes the project, adds 2 teammates as collaborators.

b Each teammate: **clone**, make a branch, change a **different** feature, push, open a **PR**.

c Review each other's PR: leave 1 question and 1 suggestion. Approve, then merge.

d Now both edit the **same line** on purpose. Resolve the conflict together.

e Write a README: what the project does, wiring, how to run it.



Check: Your repo shows 2 merged PRs, 1 resolved conflict and a README.

MAKE

Test like an engineer

Repo health check

Check	Done?	Notes
Commits with 'why' messages		
Branches used for features		
PRs reviewed + merged		
Conflict resolved		
.gitignore + no secrets		

Debug checklist

'rejected: fetch first'? Someone pushed before you: git pull, then push.
Stuck in a vim screen after merge? Type :wq and Enter.
Committed a secret? Tell the teacher: change the password; deleting the file is NOT enough.

If you're scared you'll break something: make a branch. Branches are free.

MAKE

Level up!

Team engineer challenges

Choose your challenge

Issues: create GitHub Issues for bugs and link PRs that fix them (Fixes #3).

Actions: add a GitHub Action that checks the Arduino code compiles on every PR.

Rebase: learn git rebase vs merge. When is each better?

Open source: find a 'good first issue' on an Arduino library and fix a typo in its docs.

Think like an engineer

Why do teams require a review before merging into main?

Boss level

Turn your capstone project into a public portfolio repo with photos, a demo GIF and a clear README.

Objective check

Can I really do it? Prove it.

01	Explain commit, staging and history with your own repo.	○ ○ ○
02	Show git log: are your messages meaningful?	○ ○ ○
03	Show a merge conflict you resolved and explain your choice.	○ ○ ○
04	Show a reviewed, merged pull request from your team.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Repo tour: each team shows its GitHub history like a story: what happened, who did it, and why.

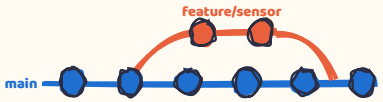
Show what you know

What did we learn?

- [x] Commits are snapshots with a 'why'
- [x] edit → status → diff → add → commit
- [x] Branches for features; merge when ready
- [x] Conflicts = same lines changed: you decide
- [x] Pull before work, push after; review PRs; no secrets

Exit ticket: what's the difference between git add and git commit?

At home
Make a repo for one of your school projects (even notes) and commit a change every day for a week.



Next class:
Smart Microgrid Model!